Authx

class authx.main.AuthX

```
AuthX(config=AuthXConfig(), model=None)
```

Bases: `_CallbackHandler[T]`, `_ErrorHandler`

The base class for AuthX.

AuthX enables JWT management within a FastAPI application. Its main purpose is to provide a reusable & simple syntax to protect API with JSON Web Token authentication.

PARAMETER	DESCRIPTION
`config`	Configuration instance to use. Defaults to AuthXConfig(). **TYPE:** `AuthXConfig` **DEFAULT:** `AuthXConfig()`
`model`	Model type hint. Defaults to dict[str, Any]. **TYPE:** `Optional[T]` **DEFAULT:** `None`

Note

AuthX is a Generic python object. Its TypeVar is not mandatory but helps type hinting furing development

AuthX base object.

PARAMETER	DESCRIPTION
`config`	Configuration instance to use. Defaults to AuthXConfig(). **TYPE:** `AuthXConfig` **DEFAULT:** `AuthXConfig()`
`model`	Model type hint. Defaults to dict[str, Any]. **TYPE:** `Optional[T]` **DEFAULT:** `None`

Source code in `authx/main.py`

```
def __init__(self, config: AuthXConfig = AuthXConfig(), model: Optional[T] = None) -> None:
    """AuthX base object.

    Args:
        config (AuthXConfig, optional): Configuration instance to use. Defaults to AuthXConfig().
        model (Optional[T], optional): Model type hint. Defaults to dict[str, Any].
    """
    self.model: Union[T, dict[str, Any]] = model if model is not None else {}
    super().__init__(model=model)
    super(_CallbackHandler, self).__init__()
    self._config = config
```

attr model `instance-attribute`

```
model = model if model is not None else {}
```

attr config `property`

```
config
```

AuthX Configuration getter.

RETURNS	DESCRIPTION
`AuthXConfig`	Configuration BaseSettings **TYPE:** `AuthXConfig`

attr DEPENDENCY `property`

```
DEPENDENCY
```

FastAPI Dependency to return an AuthX sub-object within the route context.

attr BUNDLE `property`

```
BUNDLE
```

FastAPI Dependency to return a AuthX sub-object within the route context.

attr FRESH_REQUIRED `property`

```
FRESH_REQUIRED
```

FastAPI Dependency to enforce valid token availability in request.

attr ACCESS_REQUIRED `property`

```
ACCESS_REQUIRED
```

FastAPI Dependency to enforce presence of an `access` token in request.

attr REFRESH_REQUIRED `property`

```
REFRESH_REQUIRED
```

FastAPI Dependency to enforce presence of a `refresh` token in request.

attr **ACCESS_TOKEN** `property`

```
ACCESS_TOKEN
```

FastAPI Dependency to retrieve access token from request.

attr **REFRESH_TOKEN** `property`

```
REFRESH_TOKEN
```

FastAPI Dependency to retrieve refresh token from request.

attr **CURRENT_SUBJECT** `property`

```
CURRENT_SUBJECT
```

FastAPI Dependency to retrieve the current subject from request.

attr **WS_AUTH_REQUIRED** `property`

```
WS_AUTH_REQUIRED
```

FastAPI Dependency to enforce valid access token on a WebSocket connection.

Extracts the token from the `token` query parameter or the `Authorization` header of the WebSocket handshake request.

attr **fresh_token_required** `property`

```
fresh_token_required
```

FastAPI Dependency to enforce presence of a `fresh` `access` token in request.

attr **access_token_required** `property`

```
access_token_required
```

FastAPI Dependency to enforce presence of an `access` token in request.

attr **refresh_token_required** `property`

```
refresh_token_required
```

FastAPI Dependency to enforce presence of a `refresh` token in request.

attr **MSG_TokenError** `class-attribute` `instance-attribute`

```
MSG_TokenError = 'Token Error'
```

attr **MSG_MissingTokenError** `class-attribute` `instance-attribute`

```
MSG_MissingTokenError = 'Missing JWT in request'
```

attr **MSG_MissingCSRFTokenError** `class-attribute` `instance-attribute`

```
MSG_MissingCSRFTokenError = None
```

attr **MSG_TokenTypeError** `class-attribute` `instance-attribute`

```
MSG_TokenTypeError = 'Bad token type'
```

attr **MSG_RevokedTokenError** `class-attribute` `instance-attribute`

```
MSG_RevokedTokenError = 'Invalid token'
```

attr **MSG_TokenRequiredError** `class-attribute` `instance-attribute`

```
MSG_TokenRequiredError = 'Token required'
```

attr **MSG_FreshTokenRequiredError** `class-attribute` `instance-attribute`

```
MSG_FreshTokenRequiredError = 'Fresh token required'
```

attr **MSG_AccessTokenRequiredError** `class-attribute` `instance-attribute`

```
MSG_AccessTokenRequiredError = 'Access token required'
```

attr **MSG_RefreshTokenRequiredError** `class-attribute` `instance-attribute`

```
MSG_RefreshTokenRequiredError = 'Refresh token required'
```

attr **MSG_CSRFError** `class-attribute` `instance-attribute`

```
MSG_CSRFError = 'CSRF double submit does not match'
```

attr **MSG_JWTDecodeError** `class-attribute` `instance-attribute`

```
MSG_JWTDecodeError = 'Invalid Token'
```

attr **MSG_InsufficientScopeError** `class-attribute` `instance-attribute`

```
MSG_InsufficientScopeError = None
```

attr **callback_get_model_instance** `instance-attribute`

```
callback_get_model_instance = None
```

attr **callback_is_token_in_blocklist** `instance-attribute`

```
callback_is_token_in_blocklist = None
```

attr **is_model_callback_set** `property`

```
is_model_callback_set
```

Check if callback is set for model instance.

attr **is_token_callback_set** `property`

```
is_token_callback_set
```

Check if callback is set for token.

`meth` load_config

```
load_config(config)
```

Load and store the configuration for the authentication system.

Sets the internal configuration object with the provided authentication configuration.

PARAMETER	DESCRIPTION
`config`	The configuration settings for the AuthX authentication system. **TYPE:** `AuthXConfig`

RETURNS	DESCRIPTION
`None`	None

Source code in `authx/main.py`

```
def load_config(self, config: AuthXConfig) -> None:
    """Load and store the configuration for the authentication system.

    Sets the internal configuration object with the provided authentication configuration.

    Args:
        config: The configuration settings for the AuthX authentication system.

    Returns:
        None
    """
    self._config = config
```

`meth` get_access_token_from_request `async`

```
get_access_token_from_request(request, locations=None)
```

Dependency to retrieve access token from request.

PARAMETER	DESCRIPTION
`request`	Request to retrieve access token from **TYPE:** `Request`
`locations`	Locations to retrieve token from. Defaults to None. **TYPE:** `Optional[TokenLocations]` **DEFAULT:** `None`

RAISES	DESCRIPTION
`MissingTokenError`	When no `access` token is available in request

RETURNS	DESCRIPTION
`RequestToken`	Request Token instance for `access` token type **TYPE:** `RequestToken`

Source code in `authx/main.py`

```
310 async def get_access_token_from_request(
311     self, request: Request, locations: Optional[TokenLocations] = None
312 ) -> RequestToken:
313     """Dependency to retrieve access token from request.
314
315     Args:
316         request (Request): Request to retrieve access token from
317         locations (Optional[TokenLocations], optional): Locations to retrieve token from. Defaults to
318 None.
319
320     Raises:
321         MissingTokenError: When no `access` token is available in request
322
323     Returns:
324         RequestToken: Request Token instance for `access` token type
325     """
        return await self._get_token_from_request(request, optional=False, locations=locations)
```

`meth` get_refresh_token_from_request `async`

```
get_refresh_token_from_request(request, locations=None)
```

Dependency to retrieve refresh token from request.

PARAMETER	DESCRIPTION
`request`	Request to retrieve refresh token from **TYPE:** `Request`
`locations`	Locations to retrieve token from. Defaults to None. **TYPE:** `Optional[TokenLocations]` **DEFAULT:** `None`

RAISES	DESCRIPTION
`MissingTokenError`	When no `refresh` token is available in request

RETURNS	DESCRIPTION
`RequestToken`	Request Token instance for `refresh` token type **TYPE:** `RequestToken`

Source code in `authx/main.py`

```
327 async def get_refresh_token_from_request(
328     self, request: Request, locations: Optional[TokenLocations] = None
329 ) -> RequestToken:
330     """Dependency to retrieve refresh token from request.
331
332     Args:
333         request (Request): Request to retrieve refresh token from
334         locations (Optional[TokenLocations], optional): Locations to retrieve token from. Defaults to
335 None.
336
337     Raises:
338         MissingTokenError: When no `refresh` token is available in request
339
340     Returns:
341         RequestToken: Request Token instance for `refresh` token type
342     """
    return await self._get_token_from_request(request, refresh=True, optional=False,
locations=locations)
```

`meth` verify_token

```
verify_token(token, verify_type=True, verify_fresh=False, verify_csrf=True)
```

Verify a request token.

Attempts verification with the current key first, then falls back to the previous key if key rotation is configured.

PARAMETER	DESCRIPTION
`token`	RequestToken instance **TYPE:** `RequestToken`
`verify_type`	Apply token type verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`verify_fresh`	Apply token freshness verification. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`
`verify_csrf`	Apply token CSRF verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`

RETURNS	DESCRIPTION
`TokenPayload`	Verified token payload **TYPE:** `TokenPayload`

Source code in `authx/main.py`

```python
def verify_token(
    self,
    token: RequestToken,
    verify_type: bool = True,
    verify_fresh: bool = False,
    verify_csrf: bool = True,
) -> TokenPayload:
    """Verify a request token.

    Attempts verification with the current key first, then falls back
    to the previous key if key rotation is configured.

    Args:
        token (RequestToken): RequestToken instance
        verify_type (bool, optional): Apply token type verification. Defaults to True.
        verify_fresh (bool, optional): Apply token freshness verification. Defaults to False.
        verify_csrf (bool, optional): Apply token CSRF verification. Defaults to True.

    Returns:
        TokenPayload: Verified token payload
    """
    try:
        return token.verify(
            key=self.config.public_key,
            algorithms=[self.config.JWT_ALGORITHM],
            verify_fresh=verify_fresh,
            verify_type=verify_type,
            verify_csrf=verify_csrf,
            audience=self.config.JWT_DECODE_AUDIENCE,
            issuer=self.config.JWT_DECODE_ISSUER,
        )
    except JWTDecodeError:
        previous_key = self.config.previous_public_key
        if previous_key is None:
            raise
        return token.verify(
            key=previous_key,
            algorithms=[self.config.JWT_ALGORITHM],
            verify_fresh=verify_fresh,
            verify_type=verify_type,
            verify_csrf=verify_csrf,
            audience=self.config.JWT_DECODE_AUDIENCE,
            issuer=self.config.JWT_DECODE_ISSUER,
        )
```

`meth` create_access_token

```python
create_access_token(uid, fresh=False, headers=None, expiry=None, data=None, audience=None, scopes=None, *args, **kwargs)
```

Generate an Access Token.

PARAMETER	DESCRIPTION
`uid`	Unique identifier to generate token for **TYPE:** `str`
`fresh`	Generate fresh token. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`
`headers`	Custom JWT headers. Defaults to None.

PARAMETER	DESCRIPTION
	TYPE: `Optional[dict[str, Any]]` **DEFAULT:** `None`
`expiry`	Use a user defined expiry claim. Defaults to None. **TYPE:** `Optional[DateTimeExpression]` **DEFAULT:** `None`
`data`	Additional data to store in token. Defaults to None. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`audience`	Audience claim. Defaults to None. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`scopes`	List of scopes to include in the token. Defaults to None. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`str`	Access Token **TYPE:** `str`

Example

```
# Token with scopes
token = auth.create_access_token(
    uid="user123",
    scopes=["users:read", "posts:write"]
)
```

Source code in `authx/main.py`

```python
def create_access_token(
    self,
    uid: str,
    fresh: bool = False,
    headers: Optional[dict[str, Any]] = None,
    expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    scopes: Optional[list[str]] = None,
    *args: Any,
    **kwargs: Any,
) -> str:
    """Generate an Access Token.

    Args:
        uid (str): Unique identifier to generate token for
        fresh (bool, optional): Generate fresh token. Defaults to False.
        headers (Optional[dict[str, Any]], optional): Custom JWT headers. Defaults to None.
        expiry (Optional[DateTimeExpression], optional): Use a user defined expiry claim. Defaults to None.
        data (Optional[dict[str, Any]], optional): Additional data to store in token. Defaults to None.
        audience (Optional[StringOrSequence], optional): Audience claim. Defaults to None.
        scopes (Optional[list[str]], optional): List of scopes to include in the token. Defaults to None.

    Returns:
        str: Access Token

    Example:
        ```python
        # Token with scopes
        token = auth.create_access_token(
            uid="user123",
            scopes=["users:read", "posts:write"]
        )
        ```
    """
    return self._create_token(
        uid=uid,
        type="access",
        fresh=fresh,
        headers=headers,
        expiry=expiry,
        data=data,
        audience=audience,
        scopes=scopes,
)
```

`meth` create_refresh_token

```python
create_refresh_token(uid, headers=None, expiry=None, data=None, audience=None, scopes=None, *args, **kwargs)
```

Generate a Refresh Token.

PARAMETER	DESCRIPTION
`uid`	Unique identifier to generate token for **TYPE:** `str`
`headers`	Custom JWT headers. Defaults to None. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`

PARAMETER	DESCRIPTION
`expiry`	Use a user defined expiry claim. Defaults to None. **TYPE:** `Optional[DateTimeExpression]` **DEFAULT:** `None`
`data`	Additional data to store in token. Defaults to None. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`audience`	Audience claim. Defaults to None. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`scopes`	List of scopes to include in the token. Defaults to None. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`str`	Refresh Token **TYPE:** `str`

Source code in `authx/main.py`

```python
def create_refresh_token(
    self,
    uid: str,
    headers: Optional[dict[str, Any]] = None,
    expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    scopes: Optional[list[str]] = None,
    *args: Any,
    **kwargs: Any,
) -> str:
    """Generate a Refresh Token.

    Args:
        uid (str): Unique identifier to generate token for
        headers (Optional[dict[str, Any]], optional): Custom JWT headers. Defaults to None.
        expiry (Optional[DateTimeExpression], optional): Use a user defined expiry claim. Defaults to
None.
        data (Optional[dict[str, Any]], optional): Additional data to store in token. Defaults to
None.
        audience (Optional[StringOrSequence], optional): Audience claim. Defaults to None.
        scopes (Optional[list[str]], optional): List of scopes to include in the token. Defaults to
None.

    Returns:
        str: Refresh Token
    """
    return self._create_token(
        uid=uid,
        type="refresh",
        headers=headers,
        expiry=expiry,
        data=data,
        audience=audience,
        scopes=scopes,
    )
```

`meth` create_token_pair

```
create_token_pair(uid, fresh=False, headers=None, access_expiry=None, refresh_expiry=None,
data=None, audience=None, access_scopes=None, refresh_scopes=None)
```

Generate an access and refresh token pair.

Convenience method that creates both tokens at once and returns them in a standardized `TokenResponse` model.

PARAMETER	DESCRIPTION
`uid`	Unique identifier of the user. **TYPE:** `str`
`fresh`	Whether the access token should be marked as fresh. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`
`headers`	Optional custom JWT headers applied to both tokens. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`access_expiry`	Optional expiry override for the access token. **TYPE:** `Optional[DateTimeExpression]` **DEFAULT:** `None`
`refresh_expiry`	Optional expiry override for the refresh token. **TYPE:** `Optional[DateTimeExpression]` **DEFAULT:** `None`
`data`	Optional additional data stored in both tokens. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`audience`	Optional audience claim for both tokens. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`access_scopes`	Optional scopes for the access token. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`
`refresh_scopes`	Optional scopes for the refresh token. **TYPE:** `Optional[list[str]]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`TokenResponse`	A model containing `access_token`, `refresh_token`, and `token_type`. **TYPE:** `TokenResponse`

Example

```
tokens = auth.create_token_pair(uid="user123", fresh=True)
return tokens  # {"access_token": "...", "refresh_token": "...", "token_type": "bearer"}
```

Source code in `authx/main.py`

```
def create_token_pair(
    self,
    uid: str,
    fresh: bool = False,
    headers: Optional[dict[str, Any]] = None,
    access_expiry: Optional[DateTimeExpression] = None,
    refresh_expiry: Optional[DateTimeExpression] = None,
    data: Optional[dict[str, Any]] = None,
    audience: Optional[StringOrSequence] = None,
    access_scopes: Optional[list[str]] = None,
    refresh_scopes: Optional[list[str]] = None,
) -> TokenResponse:
    """Generate an access and refresh token pair.

    Convenience method that creates both tokens at once and returns them
    in a standardized ``TokenResponse`` model.

    Args:
        uid: Unique identifier of the user.
        fresh: Whether the access token should be marked as fresh. Defaults to False.
        headers: Optional custom JWT headers applied to both tokens.
        access_expiry: Optional expiry override for the access token.
        refresh_expiry: Optional expiry override for the refresh token.
        data: Optional additional data stored in both tokens.
        audience: Optional audience claim for both tokens.
        access_scopes: Optional scopes for the access token.
        refresh_scopes: Optional scopes for the refresh token.

    Returns:
        TokenResponse: A model containing ``access_token``, ``refresh_token``, and ``token_type``.

    Example:
        ```python
        tokens = auth.create_token_pair(uid="user123", fresh=True)
        return tokens  # {"access_token": "...", "refresh_token": "...", "token_type": "bearer"}
        ```
    """
    access_token = self.create_access_token(
        uid=uid,
        fresh=fresh,
        headers=headers,
        expiry=access_expiry,
        data=data,
        audience=audience,
        scopes=access_scopes,
)
    refresh_token = self.create_refresh_token(
        uid=uid,
        headers=headers,
        expiry=refresh_expiry,
        data=data,
        audience=audience,
        scopes=refresh_scopes,
)
    return TokenResponse(access_token=access_token, refresh_token=refresh_token)
```

`meth` set_access_cookies

```
set_access_cookies(token, response, max_age=None)
```

Add 'Set-Cookie' for access token in response header.

PARAMETER	DESCRIPTION
`token`	Access token **TYPE:** `str`
`response`	response to set cookie on **TYPE:** `Response`
`max_age`	Max Age cookie parameter. Defaults to None. **TYPE:** `Optional[int]` **DEFAULT:** `None`

Source code in `authx/main.py`

```
def set_access_cookies(
    self,
    token: str,
    response: Response,
    max_age: Optional[int] = None,
) -> None:
    """Add 'Set-Cookie' for access token in response header.

    Args:
        token (str): Access token
        response (Response): response to set cookie on
        max_age (Optional[int], optional): Max Age cookie parameter. Defaults to None.
    """
    self._set_cookies(token=token, type="access", response=response, max_age=max_age)
```

`meth` set_refresh_cookies

```
set_refresh_cookies(token, response, max_age=None)
```

Add 'Set-Cookie' for refresh token in response header.

PARAMETER	DESCRIPTION
`token`	Refresh token **TYPE:** `str`
`response`	response to set cookie on **TYPE:** `Response`
`max_age`	Max Age cookie parameter. Defaults to None. **TYPE:** `Optional[int]` **DEFAULT:** `None`

Source code in `authx/main.py`

```
def set_refresh_cookies(
    self,
    token: str,
    response: Response,
    max_age: Optional[int] = None,
) -> None:
    """Add 'Set-Cookie' for refresh token in response header.

    Args:
        token (str): Refresh token
        response (Response): response to set cookie on
        max_age (Optional[int], optional): Max Age cookie parameter. Defaults to None.
    """
    self._set_cookies(token=token, type="refresh", response=response, max_age=max_age)
```

`meth` unset_access_cookies

```
unset_access_cookies(response)
```

Remove 'Set-Cookie' for access token in response header.

PARAMETER	DESCRIPTION
`response`	response to remove cooke from **TYPE:** `Response`

Source code in `authx/main.py`

```
def unset_access_cookies(
    self,
    response: Response,
) -> None:
    """Remove 'Set-Cookie' for access token in response header.

    Args:
        response (Response): response to remove cooke from
    """
    self._unset_cookies("access", response=response)
```

`meth` unset_refresh_cookies

```
unset_refresh_cookies(response)
```

Remove 'Set-Cookie' for refresh token in response header.

PARAMETER	DESCRIPTION
`response`	response to remove cooke from **TYPE:** `Response`

Source code in `authx/main.py`

```python
def unset_refresh_cookies(
    self,
    response: Response,
) -> None:
    """Remove 'Set-Cookie' for refresh token in response header.

    Args:
        response (Response): response to remove cooke from
    """
    self._unset_cookies("refresh", response=response)
```

`meth` unset_cookies

```python
unset_cookies(response)
```

Remove 'Set-Cookie' for tokens from response headers.

PARAMETER	DESCRIPTION
`response`	response to remove token cookies from **TYPE:** `Response`

Source code in `authx/main.py`

```python
def unset_cookies(
    self,
    response: Response,
) -> None:
    """Remove 'Set-Cookie' for tokens from response headers.

    Args:
        response (Response): response to remove token cookies from
    """
    self.unset_access_cookies(response)
    self.unset_refresh_cookies(response)
```

`meth` get_dependency

```python
get_dependency(request, response)
```

FastAPI Dependency to return a AuthX sub-object within the route context.

PARAMETER	DESCRIPTION
`request`	Request context managed by FastAPI **TYPE:** `Request`
`response`	Response context managed by FastAPI **TYPE:** `Response`

Note

The AuthXDeps is a utility class, to enable quick token operations within the route logic. It provides methods to avoid additional code in your route that would be outside of the route logic

Such methods includes setting and unsetting cookies without the need to generate a response object beforehand

RETURNS	DESCRIPTION
`AuthXDeps`	The contextful AuthX object **TYPE:** `AuthXDependency[Any]`

Source code in `authx/main.py`

```
def get_dependency(self, request: Request, response: Response) -> AuthXDependency[Any]:
    """FastAPI Dependency to return a AuthX sub-object within the route context.

    Args:
        request (Request): Request context managed by FastAPI
        response (Response): Response context managed by FastAPI

    Note:
        The AuthXDeps is a utility class, to enable quick token operations
        within the route logic. It provides methods to avoid additional code
        in your route that would be outside of the route logic

        Such methods includes setting and unsetting cookies without the need
        to generate a response object beforehand

    Returns:
        AuthXDeps: The contextful AuthX object
    """
    return AuthXDependency(self, request=request, response=response)
```

`meth` token_required

```
token_required(type='access', verify_type=True, verify_fresh=False, verify_csrf=None,
locations=None)
```

Dependency to enforce valid token availability in request.

PARAMETER	DESCRIPTION
`type`	Require a given token type. Defaults to "access". **TYPE:** `str` **DEFAULT:** `'access'`
`verify_type`	Apply type verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`verify_fresh`	Require token freshness. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`
`verify_csrf`	Enable CSRF verification. Defaults to None.

PARAMETER	DESCRIPTION
	TYPE: `Optional[bool]` **DEFAULT:** `None`
`locations`	Locations to retrieve token from. Defaults to None. **TYPE:** `Optional[TokenLocations]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`Callable[[Request], Awaitable[TokenPayload]]`	Callable[[Request], TokenPayload]: Dependency for Valid token Payload retrieval

Source code in `authx/main.py`

```python
def token_required(
    self,
    type: str = "access",
    verify_type: bool = True,
    verify_fresh: bool = False,
    verify_csrf: Optional[bool] = None,
    locations: Optional[TokenLocations] = None,
) -> Callable[[Request], Awaitable[TokenPayload]]:
    """Dependency to enforce valid token availability in request.

    Args:
        type (str, optional): Require a given token type. Defaults to "access".
        verify_type (bool, optional): Apply type verification. Defaults to True.
        verify_fresh (bool, optional): Require token freshness. Defaults to False.
        verify_csrf (Optional[bool], optional): Enable CSRF verification. Defaults to None.
        locations (Optional[TokenLocations], optional): Locations to retrieve token from. Defaults to
None.

    Returns:
        Callable[[Request], TokenPayload]: Dependency for Valid token Payload retrieval
    """

    async def _auth_required(request: Request) -> Any:
        return await self._auth_required(
            request=request,
            type=type,
            verify_csrf=verify_csrf,
            verify_type=verify_type,
            verify_fresh=verify_fresh,
            locations=locations,
        )

    return _auth_required
```

`meth` scopes_required

```python
scopes_required(*scopes, all_required=True, verify_type=True, verify_fresh=False, verify_csrf=None,
locations=None)
```

Dependency to enforce required scopes in token.

Creates a FastAPI dependency that validates that the token contains the required scopes. Supports both simple and hierarchical scopes with wildcard matching (e.g., "admin:*" matches "admin:users").

PARAMETER	DESCRIPTION
`*scopes`	Variable number of scope strings required. **TYPE:** `str` **DEFAULT:** `()`
`all_required`	If True (default), ALL scopes must be present (AND logic). If False, at least ONE scope must be present (OR logic). **TYPE:** `bool` **DEFAULT:** `True`
`verify_type`	Apply token type verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`verify_fresh`	Require token freshness. Defaults to False. **TYPE:** `bool` **DEFAULT:** `False`
`verify_csrf`	Enable CSRF verification. Defaults to None (uses config). **TYPE:** `Optional[bool]` **DEFAULT:** `None`
`locations`	Locations to retrieve token from. Defaults to None. **TYPE:** `Optional[TokenLocations]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`Callable[[Request], Awaitable[TokenPayload]]`	Callable[[Request], Awaitable[TokenPayload]]: Dependency for scope validation.

RAISES	DESCRIPTION
`InsufficientScopeError`	When token lacks required scopes.

Example

```python
# Require single scope
@app.get("/users", dependencies=[Depends(auth.scopes_required("users:read"))])
async def list_users(): ...

# Require multiple scopes (AND)
@app.delete("/users/{id}", dependencies=[Depends(auth.scopes_required("users:read", "users:delete"))])
async def delete_user(id: int): ...

# Require any of the scopes (OR)
@app.get("/admin", dependencies=[Depends(auth.scopes_required("admin", "superuser",
all_required=False))])
async def admin_panel(): ...

# Wildcard scope
@app.get("/admin/users", dependencies=[Depends(auth.scopes_required("admin:*"))])
async def admin_users(): ...
```

Source code in `authx/main.py`

```python
def scopes_required(
    self,
    *scopes: str,
    all_required: bool = True,
    verify_type: bool = True,
    verify_fresh: bool = False,
    verify_csrf: Optional[bool] = None,
    locations: Optional[TokenLocations] = None,
) -> Callable[[Request], Awaitable[TokenPayload]]:
    """Dependency to enforce required scopes in token.

    Creates a FastAPI dependency that validates that the token contains
    the required scopes. Supports both simple and hierarchical scopes
    with wildcard matching (e.g., "admin:*" matches "admin:users").

    Args:
        *scopes: Variable number of scope strings required.
        all_required: If True (default), ALL scopes must be present (AND logic).
                      If False, at least ONE scope must be present (OR logic).
        verify_type: Apply token type verification. Defaults to True.
        verify_fresh: Require token freshness. Defaults to False.
        verify_csrf: Enable CSRF verification. Defaults to None (uses config).
        locations: Locations to retrieve token from. Defaults to None.

    Returns:
        Callable[[Request], Awaitable[TokenPayload]]: Dependency for scope validation.

    Raises:
        InsufficientScopeError: When token lacks required scopes.

    Example:
        ```python
        # Require single scope
        @app.get("/users", dependencies=[Depends(auth.scopes_required("users:read"))])
        async def list_users(): ...

        # Require multiple scopes (AND)
        @app.delete("/users/{id}", dependencies=[Depends(auth.scopes_required("users:read",
"users:delete"))])
        async def delete_user(id: int): ...

        # Require any of the scopes (OR)
        @app.get("/admin", dependencies=[Depends(auth.scopes_required("admin", "superuser",
all_required=False))])
        async def admin_panel(): ...

        # Wildcard scope
        @app.get("/admin/users", dependencies=[Depends(auth.scopes_required("admin:*"))])
        async def admin_users(): ...
        ```
    """
    required_scopes = list(scopes)

    async def _scopes_required(request: Request) -> TokenPayload:
        payload = await self._auth_required(
            request=request,
            type="access",
            verify_type=verify_type,
            verify_fresh=verify_fresh,
            verify_csrf=verify_csrf,
            locations=locations,
)

        if not has_required_scopes(required_scopes, payload.scopes, all_required=all_required):
            raise InsufficientScopeError(required=required_scopes, provided=payload.scopes)

        return payload

    return _scopes_required
```

meth get_current_subject async

```
get_current_subject(request)
```

Retrieve the currently authenticated subject from the request.

Validates the request token and fetches the corresponding subject based on the user identifier.

PARAMETER	DESCRIPTION
`request`	The HTTP request containing authentication credentials. **TYPE:** `Request`

RETURNS	DESCRIPTION
`Optional[T]`	The authenticated subject if present, otherwise None.

Source code in `authx/main.py`

```
async def get_current_subject(self, request: Request) -> Optional[T]:
    """Retrieve the currently authenticated subject from the request.

    Validates the request token and fetches the corresponding subject based on the user identifier.

    Args:
        request: The HTTP request containing authentication credentials.

    Returns:
        The authenticated subject if present, otherwise None.
    """
    token: TokenPayload = await self._auth_required(request=request)
    uid = token.sub
    return await self._get_current_subject(uid=uid)
```

meth get_token_from_request async

```
get_token_from_request(request: Request, type: TokenType = 'access', optional: Literal[True] = True,
locations: Optional[TokenLocations] = None) -> Optional[RequestToken]
```

```
get_token_from_request(request: Request, type: TokenType = 'access', optional: Literal[False] =
False, locations: Optional[TokenLocations] = None) -> RequestToken
```

```
get_token_from_request(request, type='access', optional=True, locations=None)
```

Retrieve token from request.

PARAMETER	DESCRIPTION
`request`	The FastAPI request object. **TYPE:** `Request`

PARAMETER	DESCRIPTION
`type`	The type of token to retrieve from request. Defaults to "access". **TYPE:** `TokenType` **DEFAULT:** `'access'`
`optional`	Whether or not to enforce token presence in request. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`
`locations`	Locations to retrieve token from. Defaults to None (uses configured JWT_TOKEN_LOCATION). **TYPE:** `Optional[TokenLocations]` **DEFAULT:** `None`

Note

When `optional=True`, the return value might be `None` if no token is available in request.

When `optional=False`, raises a MissingTokenError.

RETURNS	DESCRIPTION
`Optional[RequestToken]`	Optional[RequestToken]: The RequestToken if available, None if optional and not found.

Example

```
token = await auth.get_token_from_request(request)
token = await auth.get_token_from_request(request, type="refresh")
token = await auth.get_token_from_request(request, optional=False)
```

Source code in `authx/main.py`

```python
async def get_token_from_request(
    self,
    request: Request,
    type: TokenType = "access",
    optional: bool = True,
    locations: Optional[TokenLocations] = None,
) -> Optional[RequestToken]:
    """Retrieve token from request.

    Args:
        request (Request): The FastAPI request object.
        type (TokenType, optional): The type of token to retrieve from request.
            Defaults to "access".
        optional (bool, optional): Whether or not to enforce token presence in request.
            Defaults to True.
        locations (Optional[TokenLocations], optional): Locations to retrieve token from.
            Defaults to None (uses configured JWT_TOKEN_LOCATION).

    Note:
        When `optional=True`, the return value might be `None`
        if no token is available in request.

        When `optional=False`, raises a MissingTokenError.

    Returns:
        Optional[RequestToken]: The RequestToken if available, None if optional and not found.

    Example:
        ```python
        token = await auth.get_token_from_request(request)
        token = await auth.get_token_from_request(request, type="refresh")
        token = await auth.get_token_from_request(request, optional=False)
        ```
    """
    if optional:
        return await self._get_token_from_request(
            request,
            locations=locations,
            refresh=(type == "refresh"),
            optional=True,
)
    else:
        return await self._get_token_from_request(
            request,
            locations=locations,
            refresh=(type == "refresh"),
            optional=False,
)
```

`meth` implicit_refresh_middleware `async`

```python
implicit_refresh_middleware(request, call_next)
```

FastAPI Middleware to enable token refresh for an APIRouter.

PARAMETER	DESCRIPTION
`request`	Incoming request **TYPE:** `Request`
`call_next`	Endpoint logic to be called **TYPE:** `Coroutine`

Note

This middleware is only based on `access` tokens. Using implicit refresh mechanism makes use of `refresh` tokens unnecessary.

Note

The refreshed `access` token will not be considered as `fresh`

Note

The implicit refresh mechanism is only enabled for authorization through cookies.

RETURNS	DESCRIPTION
`Response`	Response with update access token cookie if relevant **TYPE:** `Response`

Source code in `authx/main.py`

```python
async def implicit_refresh_middleware(
    self,
    request: Request,
    call_next: Callable[[Request], Coroutine[Any, Any, Response]],
) -> Response:
    """FastAPI Middleware to enable token refresh for an APIRouter.

    Args:
        request (Request): Incoming request
        call_next (Coroutine): Endpoint logic to be called

    Note:
        This middleware is only based on `access` tokens.
        Using implicit refresh mechanism makes use of `refresh`
        tokens unnecessary.

    Note:
        The refreshed `access` token will not be considered as
        `fresh`

    Note:
        The implicit refresh mechanism is only enabled
        for authorization through cookies.

    Returns:
        Response: Response with update access token cookie if relevant
    """
    response = await call_next(request)

    if self.config.has_location("cookies") and self._implicit_refresh_enabled_for_request(request):
        with contextlib.suppress(AuthXException):
            # Refresh mechanism
            token = await self._get_token_from_request(
                request=request,
                locations=["cookies"],
                refresh=False,
                optional=False,
            )
            payload = self.verify_token(token, verify_fresh=False, verify_csrf=False)
            if payload.time_until_expiry < self.config.JWT_IMPLICIT_REFRESH_DELTATIME:
                new_token = self.create_access_token(uid=payload.sub, fresh=False,
data=payload.extra_dict)
                self.set_access_cookies(new_token, response=response)
    return response
```

`meth` handle_errors

```python
handle_errors(app)
```

Add the `FastAPI.exception_handlers` relative to AuthX exceptions.

PARAMETER	DESCRIPTION
`app`	the FastAPI application to handle errors for **TYPE:** `FastAPI`

Source code in `authx/_internal/_error.py`

```python
def handle_errors(self, app: FastAPI) -> None:
    """Add the `FastAPI.exception_handlers` relative to AuthX exceptions.

    Args:
        app (FastAPI): the FastAPI application to handle errors for
    """
    self._set_app_exception_handler(app, exception=exceptions.JWTDecodeError, status_code=422,
message=None)
    self._set_app_exception_handler(
        app,
        exception=exceptions.MissingTokenError,
        status_code=401,
        message=self.MSG_TokenError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.MissingCSRFTokenError,
        status_code=401,
        message=None,  # Use detailed exception message for better user guidance
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.TokenTypeError,
        status_code=401,
        message=self.MSG_TokenTypeError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.RevokedTokenError,
        status_code=401,
        message=self.MSG_RevokedTokenError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.TokenRequiredError,
        status_code=401,
        message=self.MSG_TokenRequiredError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.FreshTokenRequiredError,
        status_code=401,
        message=self.MSG_FreshTokenRequiredError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.AccessTokenRequiredError,
        status_code=401,
        message=self.MSG_AccessTokenRequiredError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.RefreshTokenRequiredError,
        status_code=401,
        message=self.MSG_RefreshTokenRequiredError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.CSRFError,
        status_code=401,
        message=self.MSG_CSRFError,
    )
    self._set_app_exception_handler(
        app,
        exception=exceptions.InsufficientScopeError,
        status_code=403,
        message=None,  # Use detailed exception message showing required vs provided scopes
    )
```

`meth` **set_callback_get_model_instance**

```
set_callback_get_model_instance(callback)
```

Set callback for model instance.

Source code in `authx/_internal/_callback.py`

```
def set_callback_get_model_instance(self, callback: ModelCallback[T]) -> None:
    """Set callback for model instance."""
    self.callback_get_model_instance = callback
```

meth set_callback_token_blocklist

```
set_callback_token_blocklist(callback)
```

Set callback for token.

Source code in `authx/_internal/_callback.py`

```
def set_callback_token_blocklist(self, callback: TokenCallback) -> None:
    """Set callback for token."""
    self.callback_is_token_in_blocklist = callback
```

meth set_subject_getter

```
set_subject_getter(callback)
```

Set the callback to run for subject retrieval and serialization.

Source code in `authx/_internal/_callback.py`

```
def set_subject_getter(self, callback: ModelCallback[T]) -> None:
    """Set the callback to run for subject retrieval and serialization."""
    self.set_callback_get_model_instance(callback)
```

meth set_token_blocklist

```
set_token_blocklist(callback)
```

Set the callback to run for validation of revoked tokens.

Source code in `authx/_internal/_callback.py`

```
def set_token_blocklist(self, callback: TokenCallback) -> None:
    """Set the callback to run for validation of revoked tokens."""
    self.set_callback_token_blocklist(callback)
```

meth is_token_in_blocklist async

```
is_token_in_blocklist(token, **kwargs)
```

Check if token is in blocklist.

Source code in `authx/_internal/_callback.py`

```python
async def is_token_in_blocklist(self, token: Optional[str], **kwargs: ParamSpecKwargs) -> bool:
    """Check if token is in blocklist."""
    if self._check_token_callback_is_set(ignore_errors=True):
        callback: Optional[TokenCallback] = self.callback_is_token_in_blocklist
        if callback is not None and token is not None:
            if iscoroutinefunction(callback):
                return await callback(token, **kwargs)
            return cast(bool, callback(token, **kwargs))
    return False
```

March 20, 2026 March 19, 2026 GitHub